# Terraform Associate

## Certification Study Guide

**HashiCorp Certified: Terraform Associate (003)**

Comprehensive Exam Revision Reference

Written by

**Ahsan Ijaz**

HashiCorp Certified — Terraform Associate (003)

# Table of Contents

Table of Contents ..... 2
1. Terraform Basics ..... 4
What is Terraform? ..... 4
Declarative vs. Imperative ..... 4
The Core Terraform Workflow ..... 4
HCL Syntax Basics ..... 4
2. Providers and Resources ..... 5
Providers ..... 5
Resources ..... 6
3. Terraform CLI Commands ..... 7
Command Reference ..... 7
terraform init ..... 7
terraform plan ..... 7
terraform apply ..... 8
terraform fmt ..... 8
terraform state Subcommands ..... 8
4. Variables and Outputs ..... 9
Input Variables ..... 9
Locals ..... 10
Outputs ..... 10
5. State Management ..... 12
What is the State File? ..... 12
State Transfer After First Apply ..... 12
Remote Backends Comparison ..... 12
Backend Execution Types ..... 13
terraform -refresh-only vs. terraform refresh ..... 13
6. Modules ..... 14
Standard Module Structure ..... 14
Module Sources ..... 14
Module Version Constraints ..... 14
7. Expressions and Functions ..... 15
count vs. for_each ..... 15
String Interpolation ..... 15
Built-in Functions ..... 15

Lifecycle Validation Features ..... 16
8. Lifecycle Rules and Provisioners ..... 17
lifecycle Meta-Arguments ..... 17
Provisioners ..... 17
9. HCP Terraform and Terraform Cloud ..... 18
HCP Terraform Editions Compared ..... 18
What HCP Terraform Can Do That S3 Cannot ..... 18
Workspaces ..... 19
Execution Modes ..... 19
VCS Integration ..... 19
Policy Enforcement (Sentinel) ..... 19
Variable Sets ..... 20
Cross-Workspace Outputs ..... 20
10. Logging and Debugging ..... 21
Enabling Logging — TF_LOG ..... 21
TF_LOG Levels ..... 21
Key Environment Variables ..... 21
11. Security and Best Practices ..... 23
Protecting Backend Credentials ..... 23
Sensitive Data in State ..... 23
Version Pinning ..... 23
State Locking ..... 23
Testing with terraform test ..... 24
12. Testing and Security Tools ..... 25
13. Common Mistakes and Misconceptions ..... 26
State-Related Mistakes ..... 26
Backend Configuration Mistakes ..... 26
HCP Terraform Mistakes ..... 26
14. Last-Minute Revision Cheat Sheet ..... 27
Core Concepts at a Glance ..... 27
Key Command Summary ..... 27
Version Constraint Quick Reference ..... 28
15. Real Exam Practice Questions ..... 29

# 1. Terraform Basics

## What is Terraform?

**Terraform** is an open-source **Infrastructure as Code (IaC)** tool by HashiCorp that lets you define, provision, and manage infrastructure across multiple cloud providers using a **declarative**, human-readable configuration language called **HCL (HashiCorp Configuration Language)**.

## Declarative vs. Imperative

| Concept | Description | Example |
|---|---|---|
| Declarative | Describe the desired end state; Terraform figures out how to get there | "I want 3 EC2 instances" |
| Imperative | Describe every step to reach the goal | "Run this script to launch 3 servers" |

**EXAM TIP** Terraform is declarative. You do not script the steps — you declare the outcome.

## The Core Terraform Workflow

### Write → Plan → Apply

- **Write —** Author .tf configuration files describing desired infrastructure.
- **Plan —** Terraform computes a diff between current state and desired state.
- **Apply —** Terraform executes the plan and provisions infrastructure.

## HCL Syntax Basics

```
# Block syntax: <BLOCK_TYPE> "<TYPE>" "<NAME>"
resource "aws_instance" "web" {
  ami           = "ami-12345678"
  instance_type = "t3.micro"

  tags = {
    Name = "my-web-server"
  }
}
```

- Arguments use key = value format.
- Strings are double-quoted.
- **Comments:** `# or //` (single-line), `/* */` (multi-line).

**IMPORTANT** Terraform treats infrastructure as immutable. If a change cannot be made in-place, Terraform destroys the old resource and creates a new one.

# 2. Providers and Resources

## Providers

A provider is a plugin that tells Terraform how to communicate with a specific API (AWS, Azure, GCP, Kubernetes, etc.).

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region = "us-east-1"
}
```

- Providers are downloaded during `terraform init` into `.terraform/providers/`.
- `.terraform.lock.hcl` pins exact provider versions — **commit this to Git**.

### Provider Aliases (Multi-Region / Multi-Account)

```
provider "aws" {
  region = "us-east-1"
}

provider "aws" {
  alias  = "west"
  region = "us-west-2"
}

resource "aws_instance" "west_vm" {
  provider      = aws.west
  ami           = "ami-12345"
  instance_type = "t3.micro"
}
```

**EXAM TIP** Without an alias, Terraform assumes only one provider configuration exists. alias is the key to multi-region deployments.

## Resources

```
resource "aws_instance" "my_vm" {
  ami           = "ami-12345678"
  instance_type = "t3.micro"
}
```

**Resource address:** `<RESOURCE_TYPE>.<NAME>` e.g., **`aws_instance.my_vm`**

- **Implicit dependency —** Terraform detects it from attribute references.
- **Explicit dependency —** Use `depends_on` when Terraform cannot auto-detect it.

> **EXAM TIP** Use terraform graph to visualise the dependency graph between resources.

# 3. Terraform CLI Commands

## Command Reference

| Command | Purpose |
|---|---|
| terraform init | Initialise working directory, download providers and modules |
| terraform plan | Preview changes without applying them |
| terraform apply | Apply changes to reach the desired state |
| terraform destroy | Destroy all managed infrastructure |
| terraform validate | Check configuration files for syntax errors |
| terraform fmt | Format .tf files to the canonical style |
| terraform output | Read and display output values |
| terraform show | Display current state or saved plan (all resources) |
| terraform state | Advanced state management subcommands |
| terraform get | Download modules from remote repositories |
| terraform graph | Generate a dependency graph in DOT format |
| terraform import | Import existing infrastructure into Terraform state |
| terraform test | Execute .tftest.hcl test files and assertions |

## terraform init

```
terraform init                          # Standard initialisation
terraform init -upgrade                 # Upgrade providers to newest allowed
version
terraform init -migrate-state           # Migrate state to new backend
(preserves history)
terraform init -reconfigure             # Set up new backend WITHOUT copying old
state
terraform init -backend-config=sec.hcl  # Pass backend credentials from a
separate file
```

**COMMON MISTAKE** terraform init -upgrade upgrades providers within version constraints — it does NOT upgrade Terraform itself.

## terraform plan

```
terraform plan                          # Standard plan
terraform plan -out=plan.tfplan         # Save plan to a file
terraform plan -refresh-only            # Show how state would be updated
terraform plan -target=aws_instance.web # Plan only for a specific resource
```

**EXAM TIP** -refresh-only shows proposed state updates but saves NOTHING until terraform apply -refresh-only is explicitly run.

## terraform apply

```
terraform apply                              # Standard apply (with confirmation
prompt)
terraform apply plan.tfplan                  # Execute a saved plan file
terraform apply -auto-approve                # Skip confirmation (CI/CD)
terraform apply -target=aws_instance.web     # Apply only a specific resource
terraform apply -replace=aws_instance.web    # Force destroy and recreate a
resource
terraform apply -refresh-only                # Sync state file with real
infrastructure
```

**COMMON MISTAKE** Using -target to destroy a resource but leaving the code in place means the next terraform apply will re-create it. Use the config-driven approach instead.

## terraform fmt

```
terraform fmt              # Format files in the current directory only
terraform fmt -recursive   # Also format files in subdirectories
terraform fmt -check       # Check formatting WITHOUT making changes
terraform fmt -diff        # Show diff between current and formatted versions (no
write)
```

**EXAM TIP** terraform fmt only affects the current directory by default. Use -recursive to include subdirectories.

## terraform state Subcommands

| Subcommand | Purpose |
|---|---|
| terraform state list | List all resources tracked in state (no attribute detail) |
| terraform state show <address> | Show detailed attributes of one specific resource |
| terraform state pull | Retrieve and print current state data (read-only) |
| terraform state mv | Move / rename a resource within the same state file |
| terraform state rm | Remove a resource from state without destroying infrastructure |

**EXAM TIP** terraform show (no "state") gives a comprehensive overview of ALL resources. terraform state show requires a specific resource address.

# 4. Variables and Outputs

## Input Variables

```
variable "environment" {
  type        = string
  description = "Deployment environment"
  default     = "dev"

  validation {
    condition     = contains(["dev", "staging", "prod"], var.environment)
    error_message = "environment must be dev, staging, or prod."
  }
}
```

**Variable types:** `string, number, bool, list, map, set, object, tuple, any`

### Reserved Variable Names — Cannot Be Used

The following names are reserved by Terraform and CANNOT be used as variable names:

| Reserved Name | Why It Is Reserved |
|---|---|
| source | Used in module blocks to specify the module source path or registry address |
| version | Used in required_providers and module blocks for version constraints |
| providers | Used in module blocks to pass provider configurations |
| count | Meta-argument for creating multiple resource instances |
| for_each | Meta-argument for creating named instances from a map or set |
| lifecycle | Meta-argument block controlling resource lifecycle behaviour |
| depends_on | Meta-argument for explicit dependency declarations |
| locals | Block type for declaring local values |

**EXAM TIP** On the exam: source and version are reserved and CANNOT be used as variable names. name and data are NOT reserved and CAN be used as variable names.

### Variable Precedence (Lowest to Highest)

1. Default values inside the variable "name" {} block (Lowest Priority)
2. Environment variables (TF_VAR_name)

3. The terraform.tfvars file
4. The terraform.tfvars.json file
5. Any *.auto.tfvars or *.auto.tfvars.json files (alphabetical order)
6. Explicit -var-file flags on the command line (order typed)
7. Explicit -var flags on the command line (Highest Priority)

### Referencing Variables — Correct Syntax

Input variables are referenced using var.<name>. You CANNOT assign values inside resource blocks.

```
# CORRECT — declare variable, then reference it via var.<name>
variable "instance_count" {
  type    = number
  default = 3
}

resource "aws_instance" "web" {
  count = var.instance_count   # correct: read-only reference
}

# WRONG — you cannot write: var.instance_count = 3 inside a resource block
# var.input is a reference, NOT an assignment target
```

**EXAM TIP** Exam question: What is the correct way to reference an input variable "input" in a resource block? Answer: var.input — not var.input = 3, which is invalid HCL syntax.

## Locals

Local values are intermediate computed values that reduce repetition and improve readability.

```
locals {
  common_tags = {
    Environment = var.environment
    Project     = "my-app"
  }
  server_name = "${var.environment}-${var.region}-web"
}
```

## Outputs

```
output "instance_ip" {
  value       = aws_instance.web.public_ip
  description = "The public IP of the web server"
  sensitive   = true   # Hides value in CLI output only
}
```

**IMPORTANT** Marking an output sensitive = true hides it in CLI output but the value is STILL written to the state file in plaintext.

## Output Visibility — Root vs. Child Modules

terraform apply only displays outputs defined in the root module. Child module outputs are NOT shown automatically — they must be re-exported by the root module to be visible.

**EXAM TIP** Exam question: "Does terraform apply show outputs of both root and child modules?" — Answer: FALSE. Only root module outputs are shown automatically.

## Accessing Outputs — Syntax by Context

| Context | Syntax |
|---|---|
| Within the same module | `resource_type.name.attribute` |
| Child module output to root | `module.<name>.<output>` |
| String interpolation | `"${module.db.url}"` |
| CLI command | `terraform output <output_name>` |
| Cross-workspace (HCP Terraform) | `data.tfe_outputs.<name>.values.<output>` |

# 5. State Management

## What is the State File?

The `terraform.tfstate` file is Terraform's source of truth about managed infrastructure. It:

- Maps configuration resources to real-world objects.
- Caches resource attributes to avoid unnecessary API calls (performance).
- Tracks metadata like resource dependencies.

**COMMON MISTAKE** Without terraform.tfstate, terraform destroy cannot identify any resources. Terraform will attempt to CREATE previously managed resources as if they were new.

## State Transfer After First Apply

After running the first `terraform apply`, it is **OPTIONAL** to transfer state to a different platform or location. Use `terraform init -migrate-state` to copy existing state to a new backend.

**EXAM TIP** Exam question: After running the first terraform apply, it is _____ to transfer state to another platform. Answer: OPTIONAL — "impossible" is wrong (Terraform fully supports this), and "desirable" is a matter of opinion. "Optional" is the correct exam answer.

## Remote Backends Comparison

| Feature | Remote (HCP Terraform) | S3 (AWS) |
|---|---|---|
| Storage | Managed by HashiCorp | Managed by you (AWS) |
| Address | Organisation + workspace | Bucket + key + region |
| Authentication | Terraform API token | AWS Access Keys / IAM Roles |
| Execution | Runs on HCP infrastructure | Runs on your local machine |
| Best For | Teams using HCP for CI/CD | Teams keeping everything in AWS |
| State File Path | Abstracted (you don't see the file) | Explicit path/to/my.tfstate |

**EXAM TIP** KEY DISTINCTION: HCP Terraform executes plans and applies on HashiCorp's own managed infrastructure (remote operations). S3 is ONLY state storage — plans still run on your local machine. This is what you can do with HCP that you CANNOT do with S3 alone.

## Backend Execution Types

| Backend Type | Where Code Executes | State Storage | Key Feature |
|---|---|---|---|
| remote / cloud (HCP) | HCP Terraform cloud | HCP Terraform Platform | Remote runs, speculative plans, Sentinel policies |
| s3 / gcs / azurerm | Locally on your machine | Public Cloud Object Buckets | Requires local cloud credentials |
| consul | Locally on your machine | Consul KV Cluster | Best for air-gapped / on-premises |

**IMPORTANT** The backend block accepts LITERAL VALUES ONLY — no variables or computed values. Use -backend-config=secrets.hcl to pass dynamic credentials at init time.

## terraform -refresh-only vs. terraform refresh

| Command | What It Does | Saves State? | Changes Infra? |
|---|---|---|---|
| terraform plan -refresh-only | Shows how state would be updated | No | No |
| terraform apply -refresh-only | Syncs state with real infrastructure | Yes | No |
| terraform refresh (legacy) | Deprecated — use -refresh-only instead | Yes | No |

# 6. Modules

## Standard Module Structure

```
my-module/
├── README.md          # Required for public registry discoverability
├── main.tf            # Core resource definitions
├── variables.tf       # Input variable declarations
├── outputs.tf         # Output value declarations
└── versions.tf        # Provider / version requirements (recommended)
```

**EXAM TIP** This is the Standard Module Structure required by HashiCorp for modules published to the public registry.

## Module Sources

| Source Type | Syntax Example |
|---|---|
| Local path | `source = "./modules/web"` |
| Terraform Public Registry | `source = "hashicorp/consul/aws"` |
| Private Registry | `source = "app.terraform.io/my-org/vpc/aws"` |
| Git (public) | `source = "git::https://github.com/org/repo.git"` |
| Git (private SSH) | `source = "git::ssh://git@gitlab.example.com/org/module.git"` |

- **Public registry format:** `<NAMESPACE>/<NAME>/<PROVIDER>`
- **Private registry format:** `<HOSTNAME>/<NAMESPACE>/<NAME>/<PROVIDER>`

## Module Version Constraints

| Constraint | Meaning |
|---|---|
| ~> 1.0.4 | Allows 1.0.5, 1.0.10 — NOT 1.1.0 (patch-level only) |
| ~> 1.1 | Allows 1.2, 1.10 — NOT 2.0 (minor-level only) |
| >=1.0, <2.0 | Explicit range — any version between 1.0 and 2.0 |

**EXAM TIP** Specifying a version in a module block sourced from the public Terraform Registry is OPTIONAL but strongly recommended for reproducibility.

**IMPORTANT** A child module should NOT contain a provider {} block. If it does, you cannot use count, for_each, or depends_on on the module call. Provider configuration belongs in the root module.

# 7. Expressions and Functions

## count vs. for_each

| Feature | count | for_each |
|---|---|---|
| Input | Integer | Map or Set of strings |
| Address | `resource.name[0]` | `resource.name["key"]` |
| Best For | Simple on/off toggle | Creating multiple named instances |
| Removal Risk | Removing index 0 shifts ALL addresses | Removing one key has no side-effects |

## String Interpolation

Use ${ } syntax to embed expressions inside strings:

```
# Join a resource attribute with a hyphen and a string suffix
name = "${aws_instance.web.name}-vat"

# Using the join() function (note: different argument order)
name = join("-", [aws_instance.web.name, "vat"])

# Multi-variable interpolation
local.server_name = "${var.environment}-${var.region}-web"
```

**EXAM TIP** To concatenate a resource attribute with a string, use "${resource_type.name.attribute}-suffix". This is standard HCL interpolation. Note: concat() is for lists, not strings; the + operator does not work in HCL.

## Built-in Functions

| Function | Purpose | Example |
|---|---|---|
| join(sep, list) | Concatenate list items into a string | join("-", ["prod","us","web"]) = "prod-us-web" |
| merge(map1, map2) | Merge multiple maps | merge(var.defaults, var.overrides) |
| lookup(map, key, default) | Get map value with fallback | lookup(var.meta, "region", "us-east-1") |

| Function | Purpose | Example |
|---|---|---|
| toset(list) | Convert a list to a set | toset(var.user_names) |
| format(fmt, ...) | String formatting | format("%s-%s", var.env, var.region) |
| contains(list, value) | Check if list contains value | contains(["dev","prod"], var.env) |
| length(list) | Return length of list/map/string | length(var.users) |
| flatten(list) | Flatten nested lists | flatten([[1,2],[3,4]]) |

## Lifecycle Validation Features

| Feature | Location | When It Runs | Purpose |
|---|---|---|---|
| validation | Inside variable block | Before plan | Catch bad user input |
| precondition | Inside lifecycle block | Before creating resource | Ensure world is ready |
| postcondition | Inside lifecycle block | After creating resource | Verify resource works as expected |
| check | Top-level block | After apply (end of run) | Health monitoring — warning only, never fails run |

# 8. Lifecycle Rules and Provisioners

## lifecycle Meta-Arguments

```
resource "aws_db_instance" "main" {
  lifecycle {
    create_before_destroy = true   # Create new before destroying old
    prevent_destroy       = true   # Block accidental destruction
    ignore_changes        = [tags] # Ignore drift on specific attributes
    replace_triggered_by  = [aws_instance.web]  # Force replace when dependency
changes
  }
}
```

**EXAM TIP** create_before_destroy = true is the correct way to achieve zero-downtime during a resource replacement.

## Provisioners

**BEST PRACTICE** Avoid provisioners whenever possible. They break Terraform's declarative model. Use them only as a last resort when no provider-based solution exists.

- **`local-exec`** — Run commands on the machine running Terraform.
- **`remote-exec`** — Run commands on the remote resource.
- **`file`** — Copy files to the remote resource.

# 9. HCP Terraform and Terraform Cloud

## HCP Terraform Editions Compared

| Feature | Community (Free) | Plus / Enterprise |
|---|---|---|
| State Storage | Included | Included |
| Remote Runs | Included | Included |
| VCS Integration | Included | Included |
| Private Registry | Included | Included |
| Team Management | Up to 5 users | Unlimited teams |
| Sentinel Policies | NOT available | Full policy enforcement |
| Cost Estimation | NOT available | Available |
| Audit Logging | NOT available | Available |
| SSO / SAML | NOT available | Available |
| Self-Hosted Agents | NOT available | Available |
| Health Assessments | NOT available | Available |

**EXAM TIP** EXAM KEY: Sentinel policy enforcement, cost estimation, audit logging, SSO/SAML, and self-hosted agents are Enterprise/Plus-ONLY features. Community (Free) is for state storage, remote runs, VCS integration, and private registry only.

## What HCP Terraform Can Do That S3 Cannot

**This is a direct exam topic.** The fundamental difference between HCP Terraform and a simple S3 backend is that HCP Terraform executes your `terraform plan` and `terraform apply` commands on **HashiCorp's own managed cloud infrastructure** — not on your local machine.

| Capability | HCP Terraform (Remote) | S3 Backend |
|---|---|---|
| State Storage | Yes | Yes |
| State Locking | Yes | Yes (via DynamoDB) |
| Execute plan/apply remotely on HCP infra | YES — this is the key difference | No — runs locally |
| Sentinel Policy Enforcement | Yes (Plus/Enterprise) | No |
| Cost Estimation | Yes (Plus/Enterprise) | No |
| Speculative Plans on PRs | Yes | No |

| Capability | HCP Terraform (Remote) | S3 Backend |
|---|---|---|
| Full Audit Trail | Yes (Plus/Enterprise) | No |

**EXAM TIP** The answer to "What can you do with HCP Terraform that you cannot do with S3?" is: EXECUTE PLAN AND APPLY RUNS ON HCP'S OWN MANAGED INFRASTRUCTURE (remote operations). S3 is just file storage — your machine still does all the work.

## Workspaces

- Each workspace has its own isolated state file.
- Each workspace has its own Terraform version setting — separate from required_version in code.

**COMMON MISTAKE** Updating required_version in your .tf file does NOT change the Terraform version HCP uses. You must also update the workspace's Terraform Version in the HCP Terraform UI.

## Execution Modes

| Mode | Who Runs plan/apply | State by HCP? | All HCP Features? |
|---|---|---|---|
| Remote execution | HCP Terraform cloud infrastructure | Yes | Yes (full) |
| Local execution | Your local machine | Yes (sync only) | No — Sentinel, cost estimation, notifications NOT available |

## VCS Integration

- Each workspace is linked to a specific branch of a VCS repo.
- A run is queued automatically when commits are merged to the linked branch.
- A speculative plan runs when a pull request is opened; results are posted in the PR.

## Policy Enforcement (Sentinel)

| Level | Effect on Run |
|---|---|
| Advisory | Warning only — run continues regardless |
| Soft Mandatory | Run fails, but a human override is possible |
| Hard Mandatory | Run is blocked completely — no override possible |

## Variable Sets

- **Global —** Applied to all current and future workspaces in the organisation.
- **Project-specific —** Applied to all workspaces within selected projects.
- **Workspace-specific —** Applied to selected workspaces only.

## Cross-Workspace Outputs

```
data "tfe_outputs" "webserver" {
  organization = "my-org"
  workspace    = "prod-webserver"
}

# Reference:
data.tfe_outputs.webserver.values.public_ip
```

Run Triggers are configured on the destination workspace — prevents core infrastructure workspaces from accidentally forcing changes on downstream workspaces.

Health Assessments run terraform plan -refresh-only on a schedule to detect drift. They alert you but will NEVER automatically queue an apply.

# 10. Logging and Debugging

## Enabling Logging — TF_LOG

Terraform logging is controlled exclusively via the `TF_LOG` environment variable. There are **NO command-line flags** on terraform plan or terraform apply to enable logging.

```
# Enable verbose logging
export TF_LOG=TRACE
export TF_LOG_PATH="terraform-debug.log"

terraform plan

# Disable logging
unset TF_LOG
```

**EXAM TIP** Exam question: terraform plan was failing — what will enable logging? Answer: Set TF_LOG=TRACE environment variable BEFORE running the command. There is NO -log, -debug, or -verbose flag on terraform plan.

## TF_LOG Levels

| Level | What It Shows |
|---|---|
| TRACE | Everything — raw HTTP requests/responses between Terraform and the provider API |
| DEBUG | Terraform's internal processes and state calculations |
| INFO | General high-level messages about what Terraform is doing |
| WARN | Non-fatal warnings (e.g., deprecation notices) |
| ERROR | Only actual errors that cause Terraform to fail |

**COMMON MISTAKE** If TF_LOG=TRACE is set, credentials passed as environment variables can appear in log files. Handle log files with care.

## Key Environment Variables

| Variable | Purpose |
|---|---|
| TF_LOG | Set logging level (TRACE, DEBUG, INFO, WARN, ERROR) |
| TF_LOG_PATH | File path where logs are saved |

| Variable | Purpose |
|---|---|
| TF_INPUT | Disable/enable interactive prompts for missing variables |
| TF_VAR_<name> | Set a Terraform variable value from the shell |
| TF_CLI_ARGS | Automatically append flags to every Terraform command |
| TF_CLI_ARGS_<command> | Append flags to one specific command (e.g., TF_CLI_ARGS_plan) |
| TF_WORKSPACE | Override the active workspace without terraform workspace select |
| TF_IN_AUTOMATION | Remove suggested next steps — keeps CI/CD logs clean |
| TF_PLUGIN_CACHE_DIR | Shared directory for downloaded provider plugins |
| TF_DATA_DIR | Change location of .terraform/ directory |

# 11. Security and Best Practices

## Protecting Backend Credentials

| Method | Lands in Git? | Lands in .terraform/? | Lands in .tfplan files? |
|---|---|---|---|
| terraform.tfvars | No (if .gitignored) | Yes | Yes |
| -backend-config=sec.hcl | No (if .gitignored) | Yes (cached) | No |
| Environment Variables | No | No | No |

**BEST PRACTICE** Use environment variables for backend credentials in CI/CD pipelines — they never land in .terraform/ or saved plan files.

## Sensitive Data in State

**IMPORTANT** Sensitive values (passwords, tokens, keys) are always written to the state file in plaintext — even when using Vault, sensitive = true, or environment variables. Protect your state file.

**Exception:** Use the ephemeral block with password_wo attribute to generate passwords that are NOT written to state.

## Version Pinning

```
terraform {
  required_version = "~> 1.5"

  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}
```

**BEST PRACTICE** Always commit .terraform.lock.hcl to version control to ensure consistent provider versions across team members and CI runs.

## State Locking

If a lock is stuck, run **`terraform force-unlock <LOCK_ID>`** to tell the backend to drop the lock.

## Testing with terraform test

- Add `.tftest.hcl` files with run and assert blocks to define test assertions for each module.
- Run `terraform test` to execute all test files and validate modules.

# 12. Testing and Security Tools

| Tool | Type | When It Runs | What It Checks |
|---|---|---|---|
| tflint | Static (Linter) | Local / Early CI | Syntax errors, provider-specific issues, code smells |
| Checkov | Static (Security) | Local / Early CI | Security vulnerabilities, exposed secrets, cloud best practices |
| tfsec | Static (Security) | Local / Early CI | IaC security scanning — similar scope to Checkov |
| Terratest | Dynamic (Functional) | CI Pipeline | Full lifecycle (init, apply, destroy) and functional testing in Go |
| InSpec | Dynamic (Compliance) | Post-Deploy / CI | Scans live cloud resources against security and compliance baselines |
| Kitchen-Terraform | Dynamic (Integration) | CI Pipeline | Ruby-driven infrastructure testing bridging Test Kitchen and Terraform |

# 13. Common Mistakes and Misconceptions

## State-Related Mistakes

| Mistake | Correct Understanding |
|---|---|
| Leaving resource code after destroy -target | Next terraform apply will RE-CREATE the resource — use config-driven approach |
| Thinking sensitive = true keeps data out of state | It only hides values in CLI output — state file still has them in plaintext |
| Assuming .tfstate.backup is a full history | It only holds ONE previous version |
| Using terraform refresh (legacy) | Use terraform apply -refresh-only instead |
| Missing state file + running terraform destroy | Terraform has no idea what to destroy and will try to CREATE resources |

## Backend Configuration Mistakes

| Mistake | Correct Understanding |
|---|---|
| Using variables inside the backend {} block | NOT allowed — backend only accepts literal values |
| init -migrate-state vs. -reconfigure | -migrate-state copies old state; -reconfigure sets up new backend and IGNORES old state |

## HCP Terraform Mistakes

| Mistake | Correct Understanding |
|---|---|
| Updating required_version to change HCP execution version | You must ALSO update the workspace's Terraform Version in the HCP UI |
| Using local execution and expecting all HCP features | Sentinel, cost estimation, and notifications require remote execution |
| Using dynamic for cross-workspace data | Use tfe_outputs data source for cross-workspace sharing |
| Thinking S3 backend = HCP Terraform | S3 is just state storage — only HCP runs plan/apply on managed infrastructure |

# 14. Last-Minute Revision Cheat Sheet

## Core Concepts at a Glance

- Terraform is **declarative** and treats infrastructure as **immutable**.
- State file is the **source of truth** — protect it.
- `terraform init` downloads providers into `.terraform/providers/`.
- `.terraform.lock.hcl` pins provider versions — **commit this to Git**.
- Backend blocks accept **literal values only** — no variables.
- `sensitive = true` hides CLI output but **NOT state**.
- Logging uses **`TF_LOG`** env var — NO terraform plan flags for logging.
- Only root module outputs shown by `terraform apply` — not child module outputs.
- `source` and `version` are reserved — CANNOT be used as variable names.
- State migration after first apply is **OPTIONAL** — use terraform init -migrate-state.
- HCP Terraform runs plan/apply on **HashiCorp's own infrastructure** — S3 is storage only.
- `var.input` is a read reference — `var.input = 3` inside a resource block is INVALID HCL.

## Key Command Summary

| Task | Command |
|---|---|
| Initialise | `terraform init` |
| Migrate state to new backend | `terraform init -migrate-state` |
| Preview changes | `terraform plan` |
| Save a plan | `terraform plan -out=plan.tfplan` |
| Apply saved plan | `terraform apply plan.tfplan` |
| Force recreate a resource | `terraform apply -replace=<address>` |
| Sync state with reality | `terraform apply -refresh-only` |
| Destroy specific resource | `terraform destroy -target=<address>` |
| Check formatting (no write) | `terraform fmt -check` |
| Format recursively | `terraform fmt -recursive` |
| Validate syntax | `terraform validate` |
| List state resources | `terraform state list` |
| Show one resource detail | `terraform state show <address>` |
| Show all resources | `terraform show` |
| View dependency graph | `terraform graph` |
| Run tests | `terraform test` |

| Task | Command |
|---|---|
| Enable logging | `export TF_LOG=TRACE` |

## Version Constraint Quick Reference

| Constraint | Meaning |
|---|---|
| = 1.2.3 | Exactly this version only |
| != 1.2.3 | Any version except this one |
| > 1.2 | Greater than 1.2 |
| >= 1.2, < 2.0 | Explicit range |
| ~> 1.0.4 | 1.0.x only — patch-level flexibility, NOT 1.1.0 |
| ~> 1.1 | 1.x only — minor-level flexibility, NOT 2.0 |

# 15. Exam Practice Questions

Questions based on topics encountered in the HashiCorp Certified: Terraform Associate (003).

**Q1. What is the key difference between HCP Terraform Community edition and Enterprise/Plus edition?**

A. Community edition does not support remote state storage
**B. Enterprise adds Sentinel policies, audit logging, SSO/SAML, cost estimation, and self-hosted agents; Community is feature-limited**
C. Community edition cannot connect to a VCS provider
D. Enterprise supports more cloud providers

**Answer: B**

**Explanation:** Community (Free) supports state storage, remote runs, VCS integration, and a private registry. Enterprise/Plus adds Sentinel policy enforcement, cost estimation, audit logging, SSO/SAML, self-hosted agents, and advanced team management. If the exam asks about any of those premium features — the answer is Enterprise/Plus, not Community.

**Q2. What can you do with HCP Terraform that is fundamentally different from using an S3 backend?**

A. Store the state file remotely
B. Lock state during operations to prevent conflicts
**C. Execute terraform plan and apply runs on HCP's own managed infrastructure (remote operations)**
D. Encrypt the state file at rest

**Answer: C**

**Explanation:** An S3 backend is purely state storage — your plan and apply still execute on your local machine. HCP Terraform (remote backend) runs plans and applies on HashiCorp's own managed infrastructure, enabling Sentinel policy checks, speculative plans, cost estimation, and a full audit trail — none of which are possible with S3 alone.

**Q3. Which of the following CAN be used as an input variable name in Terraform? (Select all that apply) — Options: source, name, data, version**

A. source
**B. name**
**C. data**
D. version

**Answer: B and C (name, data)**

**Explanation:** source and version are reserved meta-arguments used in module and required_providers blocks — they cannot be used as variable names. name and data are NOT reserved and can be freely used as variable names. Other reserved names include: count, for_each, lifecycle, depends_on, providers, locals.

**Q4. True or False: terraform apply automatically displays outputs from both the root module and all child modules.**

A. True
**B. False**

**Answer: B — False**
**Explanation:** terraform apply only displays output values defined in the root module. Child module outputs are internal to that module and are NOT printed to the console automatically. To expose a child module output at the root, the root module must re-declare it: output "db_url" { value = module.database.url }.

**Q5. Which HCL expression correctly joins a resource's name attribute with a hyphen and the string "vat"? (result should be "myresource-vat")**

A. concat(aws_instance.web.name, "-vat")
**B. "${aws_instance.web.name}-vat"**
C. join(aws_instance.web.name, "-vat")
D. aws_instance.web.name + "-vat"

**Answer: B**
**Explanation:** HCL uses ${ } interpolation inside double-quoted strings. "${resource_type.name.attribute}-suffix" evaluates the attribute and appends the literal suffix. concat() works on lists, not strings. join() has the wrong argument order — it expects a separator and a list. The + operator is not valid HCL syntax for string concatenation.

**Q6. terraform plan is failing with no useful output. What will enable detailed logging?**

A. terraform plan -log=TRACE
B. terraform plan -debug
**C. Set the environment variable TF_LOG=TRACE before running the command**
D. terraform plan -verbose

**Answer: C**
**Explanation:** Terraform has NO command-line flags to enable logging. There is no -log, -debug, or -verbose flag on any Terraform command. Logging is controlled exclusively through the TF_LOG environment variable (set to TRACE, DEBUG, INFO, WARN, or ERROR). To use it: export TF_LOG=TRACE, then run terraform plan. Optionally set TF_LOG_PATH to write to a file.

**Q7. What is the correct way to reference an input variable named "input" (with a default value of 3) inside a resource block?**

A. input = 3
B. var.input = 3
**C. var.input**
D. variable.input

**Answer: C — var.input**

**Explanation:** Variables are referenced using var.<name> syntax. The value is set outside the resource block (in a variable default, -var flag, or .tfvars file). You never assign values inside a resource block. var.input = 3 is not valid HCL — var.input is a read reference, not an assignment target. variable.input is also not valid syntax.

**Q8. True or False: Specifying a version constraint is compulsory when using a module sourced from the public Terraform Registry.**

A. True
**B. False**

**Answer: B — False**
**Explanation:** Specifying a version for public registry modules is OPTIONAL. If no version is specified, Terraform downloads the latest version. However, it is strongly recommended as a best practice to always pin a version for reproducible and stable builds. Note: this is also true for required_providers — version is optional but recommended.

**Q9. After running the first terraform apply, it is _____ to transfer the state to a different platform or location.**

A. Impossible
**B. Optional**
C. Desirable
D. None of the above

**Answer: B — Optional**
**Explanation:** Migrating Terraform state to a different backend after the first apply is entirely optional and a completely normal operational task. Use terraform init -migrate-state to copy existing state to the new backend. It is not impossible (Terraform fully supports this). Whether it is "desirable" is subjective and context-dependent — making "Optional" the precise and correct exam answer.